

Estado y dirección del proyecto — 2026-10-02

Qué es esto. El análisis que pediste la noche del 1 de octubre: dónde está AlgoProject frente a su objetivo, qué hay que arreglar, qué mejorar, qué falta por construir, y un plan por fases.

El objetivo, con tus palabras. Una plataforma automatizada de generación de EAs de trading y de carteras, con seguimiento de su rendimiento; muy automatizada, con vigilancia muy limitada por tu parte, y que acabe controlando las instalaciones de SQX de dos servidores dedicados.

Cómo se hizo. Seis agentes leyeron el proyecto por áreas (workflow y pendientes, orquestación, ventana, cartera/fondeo/MT5, agentes y salud, rigor estadístico). Las afirmaciones que sostienen este documento las comprobé después yo contra el código y los datos; lo que no pude comprobar está en la última sección. Todo fue de solo lectura: no se arrancó ni paró ningún SQX ni se tocó código.

1 · Resumen ejecutivo

El proyecto es ancho y está bien instrumentado, pero todavía no ha producido nada. En un mes hay 270 commits, unas 105.000 líneas de Python, los 26 pasos del workflow con código, una ventana de 13 zonas, 16 tareas programadas y un catálogo de fondeo que se actualiza solo. Y sin embargo:

- **Ninguna estrategia ha cruzado la cadena por méritos.** Cero proyectos `Trade_`, cero supervivientes validadas, una sola identidad archivada (de desarrollo). La única corrida completa (USDJPY H1, 30 de septiembre) acabó en «ninguna estrategia sobrevive».
- **La automatización prueba la tubería, no el filtro.** El autopilot corre del paso 6 al 16 solo, pero sus criterios están vacíos (`rules: []` en los cinco pasos que juzgan) y su único corte es un sorteo de 5 estrategias. De 20 lanzamientos del 1 de octubre terminaron 2.
- **El final de la cadena no existe.** El autopilot para en el 16. La cartera tiene motor hasta F2 y nunca ha corrido con datos reales. Del puente a la VPS y del seguimiento en vivo no hay una línea de código, solo el encargo 35.
- **Todo cabe en una máquina y en una sesión tuya.** No hay cola de trabajos, no hay reintentos, no hay avisos fuera del escritorio, y los agentes dependen de tu suscripción personal de Claude.

Las cinco cosas que decidiría hoy, por orden:

1. **Proteger lo hecho.** Hay 384 ficheros sin commit (tres días de trabajo) y 11 commits sin subir, todo en un solo disco. Y la API de SQX escucha sin contraseña en un servidor con IP pública.
2. **Hacer que la fábrica decida.** Firmar los criterios de los pasos 6 y 8, escribir los de 10-16 y 20, y pasar antes una población de monos por toda la cadena. Sin esto, automatizar más solo produce más deprisa resultados que no significan nada.
3. **Hacer que la fábrica corra sola en esta máquina.** Cola persistente, reintentos, avisos al móvil y el autopilot hasta el paso 26. Solo Python: el LLM, únicamente para proponer ideas.
4. **Fondeo: de superviviente a cuenta.** Pool validado, cartera por probabilidad de aprobar, qué reto comprar, EAs listos para montar. En paralelo, la telemetría de la VPS, que ya puede leer las cuentas que tienes hoy.
5. **El segundo servidor, después.** Con la cola hecha, añadirlo es poner un nodo más. Antes, duplicaría una fábrica que aún no filtra.

El riesgo de fondo no es de ingeniería. Con los criterios propuestos, de 87.357 estrategias de calibración pasan 49 (0,056 %), y 39 de ellas están en una sola celda: USDJPY largo. La pregunta que decide el proyecto es si hay edge suficiente fuera de esa celda, y solo se contesta buscando en los 17 activos que nadie ha tocado. Para eso sirve la automatización.

2 · Supuestos que tomé sin preguntarte

- **Servidores:** esta máquina es uno de los dos; el segundo existe o existirá, con SQX. No sé si está contratado ni qué sistema lleva; el plan vale en ambos casos.
- **VPS Windows:** es la que describe el encargo 35, con 4-5 terminales de MT5 ya operando EAs hechos a mano. El seguimiento sale de ahí y el Linux calcula y enseña.
- **Tus puertas:** gastar dinero, los commits y poner un EA en una cuenta. Aprobar la idea no la marcaste, así que la trato como automatizable, con la regla 11 (ambigüedad) convertida en «la idea con dudas se aparca y se te pregunta, sin frenar a las demás».
- **Prioridad:** fondeo antes que Darwinex.
- **Costes del plan:** en sesiones de trabajo con Claude, no en euros. **S** = una sesión corta, **M** = una o dos sesiones largas, **L** = de tres a cinco sesiones. Son estimaciones mías.

3 · Dónde está el proyecto hoy

3.1 · La cadena, tramo a tramo

tramo	pasos	qué hay	qué falta para que corra sin ti
Idea y plantilla	1-3	Agentes <code>researchDirector</code> , <code>ideaExpert</code> , <code>templateArchitect</code> y la cola «Investigar» de la ventana	Nunca ha corrido de verdad: cero propuestas en disco y la cola solo se probó con dobles. Hoy la arrancas tú con un botón
Preflight y proyecto	4-5	<code>core.assets</code> , <code>builder --workflow</code>	Nada; funciona
Build	6	Filtros desde <code>_study.yaml</code> , paleta por <code>buildingBlocksExpert</code>	Tu visto bueno a las cifras. El suelo es 20-30 operaciones/año y tu regla dice 40-50
Embudo	7-16	El autopilot lo corre entero: 2 finales limpios en ~43 min	Criterios vacíos y modo desarrollo (5 al azar). Sin reintentos: 18 de 20 lanzamientos murieron
Las cuatro piezas	16.5-20	WFC, CSCV, superficies, WFM y lectura conjunta, contruidos	Se lanzan a mano. El paso 20 anota y no corta: «qué es pasar» está sin decidir
Cierre	21-25.5	Exposición, mapa, estructura, stop ATR, edge por coste	A mano. El 24 nunca vio una superviviente real; el 25 lee la madre sin stop; el 25.5 no existe
MT5	26	Verificación SQX ↔ MT5 por empresa, ~20 corridas, 2-3 min cada una	De una en una y con formulario. Falta el pool. Hantec falla y no se sabe por qué
Cartera	27+	Universo, parejas, máquina de reglas, búsqueda F2	F2 sin corrida real; F3-F5 y todo el camino de cuenta real, sin construir
EAs y despliegue	—	Export a MQ5, compilación y filtro de noticias	Magic, riesgo por cuenta, paquete para la VPS: no existen
Seguimiento en vivo	—	Nada, salvo lectores de un terminal local	Todo: telemetría, conciliación, deriva, avisos

3.2 · El embudo en números reales

- **Proyectos:** 99 en el registro, todos de prueba o heredados; 89 ya retirados. Solo USDJPY y XAUUSD, en M30, H1 y H4.
- **Estrategias construidas:** unas 87.000 de calibración en 23 poblaciones, más unas 1.700 en nueve corridas de plantilla.
- **Con los criterios propuestos para el paso 8:** 87.357 → 49 pasan y 779 quedan en limbo. Fuera de USDJPY largo, el rendimiento es de unas 3,8 por mil, cerca de lo que da el azar.
- **Donde corrió un filtro de verdad, la población murió en el paso 8:** 60 → 58 → 9 → 0. Todo lo que hay después del 8 solo ha visto supervivientes al azar o elegidas a mano.
- **Autopilot, 1 de octubre:** 20 carpetas de corrida, 2 llegaron a `FIN`. Cinco fallos por «recursos sin resolver» (el renombrado de feeds), cinco cancelados desde la ventana, dos por sincronía de disco, dos por errores de código, uno porque el filtro no dejó a nadie.
- **Registro de búsquedas:** 22 ficheros y 595 filas; **ninguna es de un build.** La búsqueda más grande, la genética, no cuenta en el número de pruebas.

3.3 · Módulo a módulo

módulo	tamaño	estado	lo más débil
ui/	35.700 líneas, 115 rutas	El más maduro; 13 zonas que funcionan	Trabajos solo en memoria; solo ve esta máquina; ningún aviso fuera de la pantalla
studies/	29.200	Todos los pasos de Python existen	Umbrales repartidos por estudio y sin firmar; varios sin veredicto definido
sqx/	13.200	Autoría y configuración sólidas, y sin GUI	La ruta en vivo espera sin límite de tiempo
pipeline/	—	Autopilot fase 1 hecha	Dos orquestadores más el de la ventana; <code>recipe.yaml</code> roto; sin estado propio
portfolio/	6.800	Fondeo: catálogo y ofertas maduros. Cartera: motor hasta F2	Nada corrido con datos reales; sin modelo de valor esperado
mt5/	—	Verificación del paso 26 funcionando bajo Wine	Una estrategia por corrida; nada hacia la VPS
ledger/	—	Diseño correcto del Sharpe desinflado	Incompleto: no apunta builds ni cortes del autopilot ni la WFM
engines/ , core/	6.900 (core)	Estables	—
agentes y cron	16 tareas	Corren cada noche	El <code>fixer</code> no arregla nada desde hace cinco noches; todo cuelga de tu cuenta de Claude
tests/	103 ficheros, 238 pruebas	Existen	La auditoría nocturna solo corre dos; cuatro fallaban el 29 de septiembre

4 · Qué hay que arreglar

Ordenado por lo que puede costar si no se toca.

4.1 · Riesgo de perder trabajo o control

1. **API de SQX abierta en un servidor con IP pública.** Los puertos 5050 y 5070 escuchan en `0.0.0.0` sin autenticación, y esta máquina tiene dirección pública. Cualquiera que llegue a ese puerto puede mandar `action=start` o vaciar un databank. No pude leer el cortafuegos (pide root), así que no sé si está expuesto. **Compruébalo tú hoy** y cierra 5050-5071 y 8080-8082.
2. **384 ficheros sin commit y 11 commits sin subir**, de los últimos tres días. El trabajo existe solo en este disco.
3. **Sin copia fuera del disco.** Repo, `AlgoData` y los proyectos de SQX viven en un único disco físico; no hay ninguna herramienta de copia instalada.
4. **/tmp es una partición de 3,9 GB al 92 %** y ya se llenó una vez, el 26 de septiembre.

4.2 · Lo que hace que una corrida desatendida se cuelgue o mienta

5. **La ruta en vivo espera para siempre.** `sqx/projects/live.py wait()` es un bucle sin límite de tiempo, sin mirar si el proceso vive y sin detectar la línea de error. Si la JVM muere, la corrida no acaba nunca.
6. **Cero reintentos.** El primer fallo de cualquier tipo tumba la corrida, incluido el error de fastutil que el propio proyecto documenta como pasajero.
7. **«Cero supervivientes» se registra como fallo**, cuando es un veredicto legítimo.
8. **sqx-worker.sh devuelve 0 cuando no debe:** `stop` aunque siga corriendo, `start` aunque ya estuviera arrancado.
9. **La regla «al custodio solo status» es convención.** `core.worker.wait_ready` manda `count`, y ninguna guarda lo impide: es la regla 1 esperando a dispararse.
10. **Un hecho que falta cuenta como limbo, y el limbo pasa.** Un estudio roto admite estrategias en silencio.
11. **Tres listas de lanzadores que no coinciden** en `ui/daemon/`; una no incluye la verificación de MT5, así que un test de Python puede arrancar durante ella.
12. **Tareas de cron que chocan:** el sábado a las 03:00 arrancan la auditoría y la actualización de datos; el lunes, la auditoría y el conserje. No hay candado entre ellas ni contra una corrida larga.

4.3 · Lo que hace que un resultado no signifique lo que parece

13. **El registro no cuenta lo que más busca.** Sin los builds ni los cortes del autopilot, el Sharpe desinflado sale optimista.
14. **oos1 ya es muestra de entrenamiento.** Lo leen diez pasos y con él se calibraron el 6 y el 8. `oos2` se alarga cada mes, así que la reserva es un blanco móvil.
15. **Los nulos están en otra escala** tras el arreglo de la t del 2 de octubre (OPEN #91): las tasas de falso positivo guardadas son cotas, de 2 a 4 veces altas.
16. **La curva diaria de SQX es el mínimo del día** (OPEN #88). Cinco estudios la diferencian, y el «peor día» no es el que mide una empresa de fondeo.
17. **Hantec no cuadra con MT5** (drawdown 0,292 % en SQX contra 0,228 %) y está sin diagnosticar. Mientras siga así, ninguna estrategia entra en el pool de Hantec.

18. **El paso 25 lee la madre sin su stop**, y el reprecio del spread no se ha contrastado con un retest de tick real (paso 25.5).

4.4 · Deriva de la documentación

- `CLAUDE.md` habla de «los 20 pasos», los encargos de 25 y `WORKFLOW.md` de 26.
- `WORKFLOW.md` da por bloqueados el paso 9 y las horas de sesión, y por pendientes los building blocks, cuando OPEN los cerró o ya hay agente. También cita un dossier (`criterios-pasos-6-y-8-2026-10-02.md`) que no está en la carpeta.
- OPEN #87 dice en el título «no construido» y en el cuerpo lista M0-F2 como construidos.
- El README de esta carpeta dice del director de investigación «nada construido»; el agente y el skill existen.
- El comentario del crontab del `fixer` habla de rama propia y worktree, contra la regla 12.
- `docs/SKILLS.md` no menciona `autopilot` , `workflow-start` ni `research-direct` .

5 · Qué hay que mejorar

1. **Un solo orquestador.** Hoy hay tres caminos sobre los mismos pasos: `pipeline/autopilot`, la cadena de la ventana y `pipeline/run.py` con su `recipe.yaml` roto. El autopilot gana; los otros se retiran o lo llaman.
2. **Estado propio de cada corrida.** El autopilot deduce por dónde va mirando el rail. Un `state.json` con la semilla, el corte aplicado y las instantáneas hace el reanudar exacto.
3. **Umbrales en un solo sitio.** Los de los pasos 10-17 viven en el `config.yaml` de cada estudio; deben ir a `ledger/thresholds.yaml` con quién los fijó y cuándo, como ya está el paso 26.
4. **Una sola memoria de resultados.** Hay tres sin unir: la memoria de investigación, el registro de búsquedas y `runs.csv / registry.csv`. El tablero que elige dónde investigar no lee el registro.
5. **Menos LLM donde basta código.** El `fixer` (Opus, cada noche) no cambia código desde hace cinco noches; el `documenter` añade filas que `checks.py` podría generar; el conserje y el skill `/autopilot` envuelven un proceso de Python. Y ningún log apunta cuántos tokens gasta cada uno.
6. **Las pruebas, todas las noches.** Correr las 238, no dos, y arreglar o apartar las que fallan.
7. **Exportaciones duplicadas.** `MCR_All` se exportó 12 veces en un día (4,9 GB) y `reports` pasa del presupuesto; el log del custodio llegó a 931 MB por una tormenta de excepciones.
8. **Datos al día sin ti.** Los escaneos de spread y de calidad del feed son manuales, y `AlgoData/INDEX.md` lista tres feeds cuando hay veinte.
9. **Ficheros al límite.** Ocho módulos de `ui/` están en 250 líneas justas y otros diez rozan el tope: la próxima función obliga a partirlos.
10. **Skills que sobran.** `oos-gate`, `curate` y `template-run` quedan dentro del autopilot; `audit` y `doc` son envoltorios de un agente; los dos globales de `sqx-lab` duplican los propios.

6 · Qué falta por construir

6.1 · Cola de trabajos y recuperación sin humano

Hoy: un trabajo por lanzamiento. Si el custodio está ocupado, el lanzador se niega en vez de esperar. Los trabajos de la ventana viven en memoria y mueren con el demonio.

Hace falta:

- Una cola en disco (SQLite en `AlgoData`) con un despachador por instalación, como servicio del sistema y no dentro de la ventana.
- Estados por trabajo: en cola, corriendo, terminado, cero supervivientes, fallo pasajero (reintenta), fallo duro (te avisa).
- Comprobación previa de que feeds, instrumentos y bloques resuelven antes de gastar un arranque: eso habría evitado 5 de los 18 fallos del 1 de octubre.
- Un vigilante: `estado.txt` sin cambios en N minutos, memoria de la JVM, RAM libre, disco.
- Retirada automática del proyecto `Test_` al acabar.

6.2 · Criterios congelados y contabilidad de la búsqueda

- Reglas reales en `criteria.yaml` para 8, 10, 12, 14 y 16, y `dev.on: false`. Para el paso 8 ya hay una propuesta validada sin aplicar (`AlgoData/scratch/calib_final/final_step8.yaml`).
- La definición de «pasar» del paso 20.
- **La población nula de punta a punta**, que pediste el 23 de septiembre y está aparcada. Es lo único que dice cuántas estrategias sin edge llegarían al final por azar.
- El registro completo: el builder, el autopilot y la WFM apuntan su fila solos; un N global entre estudios; una regla que use el Sharpe desinflado.
- `ALGO_AUTONOMOUS=1` cuando corre el autopilot, para que la reserva de `oos2` se respete justo cuando nadie mira.

6.3 · El autopilot hasta el final

- **Fase 2 (16.5 → 20):** variantes, WFC, CSCV, superficies, WFM y lectura conjunta, encadenados.
- **Fase 3 (21 → 26):** cierre, stop ATR con la versión con stop guardada como identidad propia, y verificación en MT5 por lotes sobre todas las supervivientes y todas las empresas.
- **Un registro de estrategias** por identidad con su ciclo de vida: candidata → pasó el paso N → verificada por empresa → en el pool → desplegada → retirada. Con el hash de criterios que dio cada veredicto, el EA compilado, su magic y su cuenta. Hoy esto no existe y es la bisagra entre la fábrica y la cartera.

6.4 · El lazo de investigación

- La primera corrida real de la cola «Investigar», de la propuesta al veredicto.
- Un temporizador que proponga y lance sin botón, con un tope de gasto por día en tokens y en CPU.
- La memoria que devuelve el veredicto al tablero: hoy el factor «pasado» es plano (0,50) porque las 23 corridas guardadas son de desarrollo y no cuentan.

- La fábrica entera en Python. El LLM queda para proponer ideas y redactar plantillas; si se acaban los tokens, lo ya propuesto sigue corriendo.

6.5 · Cartera y fondeo

- Archivado automático de supervivientes en el pool.
- F3 (intradía exacto en M1, los tres Monte Carlo, riesgo por fase) y F4 (veredicto en `oos1 + oos2`).
- F5: la simulación de caja y el valor esperado por plan. Es lo que contesta **qué reto comprar**; hoy `deals.worth` solo compara precios suponiendo edge cero.
- Un barrido de planes × complementos × ofertas, no un plan por corrida.
- Una pestaña de cartera en la ventana.

6.6 · Puente a la VPS y seguimiento en vivo

No hay código. El encargo 35 lo especifica y sigue vigente. La forma mínima:

- **En la VPS:** un servicio MQL5 por terminal, sin gráfico y sin órdenes, que cada minuto escribe equity, balance, posiciones con su flotante, estado de AutoTrading y operaciones nuevas con su magic. Una tarea de Windows empuja los ficheros por una red privada (Tailscale) y reabre los terminales caídos.
- **En el Linux:** un colector a `AlgoData/fleet/<cuenta>/`, un registro de magics (uno por estrategia × activo × timeframe) y una zona «Flota» en la ventana.
- **Tres comparaciones:**
 - *Diaria, por cuenta:* la equity contra los suelos de la empresa. Distancia a la pérdida diaria y a la máxima. Reutiliza `portfolio/funded/rules/`.
 - *Semanal, por magic:* las operaciones reales contra un retest de SQX de esos mismos días a los costes de esa empresa. Reutiliza `mt5/verify/` y sus umbrales. Es el `weeklyReconciler`.
 - *Mensual, por estrategia:* el resultado real contra la banda de ventanas de igual longitud de su propio backtest. Fuera de la banda, candidata a retirar; decides tú.
- **Despliegue:** paquete por cuenta (`.ex5` , `.set` , perfil de gráficos) generado desde la cartera. Montarlo en la cuenta sigue siendo tu puerta.

6.7 · Avisos

Hoy el único aviso es una notificación de escritorio para las ofertas de fondeo, que se pierde si no hay sesión gráfica. Hace falta un bus de eventos en disco y un canal al móvil (ntfy o Telegram) con cuatro clases de mensaje: **hay que decidir** (dinero, EA a cuenta, duda de una idea), **algo se ha roto** (fallo duro, servidor caído, disco), **una cuenta se acerca a un suelo**, y un **resumen diario** de una pantalla.

6.8 · Orquestación de dos servidores

Lo que ya ayuda: los roles se resuelven en un solo sitio (`core/paths.py`) desde `config/machine.yaml` .

Lo que estorba: el rol se trata como una carpeta local en unas 45 partes del código. La llamada HTTP es lo de menos: los cortes borran `.sqx` en disco, las instantáneas copian carpetas, el candado y los tokens se leen de ficheros, y la vida del proceso se mira en `/proc` .

Recomendación: no hacer remoto cada fichero. Cada servidor corre la cadena entera contra sus propias instalaciones y su propio `AlgoData` , con su demonio y su despachador. Por encima, un coordinador reparte trabajos de la cola y recoge resultados. Concretamente:

1. `machine.yaml` gana un nombre de nodo, y el registro de proyectos pasa a `nodo:instalación` (hoy dos `SQX_w2` chocarían).
2. El demonio de cada nodo sigue en `127.0.0.1` y se alcanza por túnel; rutas de solo lectura separadas de las que lanzan, con token en las segundas.
3. Los resultados vuelven por `rsync` al nodo principal; el registro tiene un solo escritor.
4. Los datos de barras y las plantillas se instalan en todos los nodos.
5. La ventana gana un selector de servidor y una vista «Torre»: una tarjeta por instalación.
6. Los agentes dejan de depender del binario de la extensión de VS Code y de tu sesión.

7 · Puntos de intervención humana

Los que se quedan (los tres que marcaste)

puerta	cómo debe llegarte
Gastar dinero	Aviso con el reto recomendado, su valor esperado y la oferta vigente. Sí o no
Poner un EA en una cuenta	Paquete listo y hoja de la cartera. Lo montas tú, o apruebas el despliegue
Commits	Una propuesta diaria ya agrupada por tema (existe el skill /sync). Un sí al día

Sobre los commits: mantener la puerta está bien, pero hoy el atasco tiene coste. Con más de 300 ficheros sucios, el `fixer` nocturno clasifica casi todo como «de otra sesión» y no arregla nada.

Decisiones de una sola vez que hoy bloquean

No son vigilancia: se toman una vez y la máquina las aplica siempre. Están en la sección 9.

Los que sobran

intervención de hoy	por qué sobra	qué la sustituye
Parar tras cada paso que juzga (8, 10, 12, 14, 16)	Con criterios firmados, la regla decide	El autopilot, también desde la ventana
«Continuar workflow», «Lanzar en SQX», ► por paso	El preflight ya decide si es seguro	La cola
Pulsar «Proponer investigación» y vetar ideas	No marcaste la idea como puerta tuya	Temporizador con tope de gasto
Elegir <code>Test_</code> o <code>Trade_</code> en cada proyecto	Es una política, no una decisión por caso	Regla fija: <code>Test_</code> hasta que haya criterios firmados
Archivar supervivientes a mano	Mecánico	El paso 26 escribe el pool
Verificar en MT5 de una en una, con formulario	Mecánico	Lote con perfil guardado
Lanzar a mano 16.5 → 25	Mecánico	Fases 2 y 3 del autopilot
Retirar proyectos <code>Test_</code>	Mecánico	Al acabar el trabajo de la cola
Vigilar corridas largas	—	Vigilante y avisos

Una que conviene mantener aunque no la marcaste: la primera vez que un activo nuevo entra en la búsqueda, su ficha de costes. Un coste mal puesto invalida todo lo que se construya encima.

8 · El plan

Ordenado por dependencia y, a igualdad, por valor. Cada fase deja algo que funciona.

Fase 0 — Proteger lo que hay (días)

1. **Cerrar la API de SQX al exterior.** - *Porqué:* escucha sin contraseña en una máquina con IP pública. - *Coste:* S, y es tuyo: pide root. - *Hecho:* desde fuera, los puertos 5050-5071 y 8080-8082 no responden.
2. **Commit y push de lo pendiente, y rito diario.** - *Porqué:* tres días de trabajo en un solo disco; desbloquea al `fixer`. - *Coste:* S. - *Hecho:* `git status` limpio, `origin` al día, y cada mañana una propuesta de commit agrupada.
3. **Copia nocturna fuera del disco** del repo, AlgoData (sin `raw` ni `scratch`) y los proyectos de SQX. - *Porqué:* hoy un fallo de disco se lo lleva todo. - *Coste:* S-M. Destino: el segundo servidor si ya existe, o un almacenamiento barato. - *Hecho:* una restauración de prueba recupera un proyecto y un estudio.
4. **Poner al día `WORKFLOW.md`, `OPEN #87` y `CLAUDE.md`** con las contradicciones de §4.4. - *Coste:* S. - *Hecho:* los tres dicen el mismo número de pasos y el mismo estado.

Fase 1 — Que la fábrica decida (1-2 semanas)

5. **Firmar los criterios de los pasos 6 y 8.** - *Porqué:* sin regla, el autopilot sortea. - *Coste:* S; es decisión tuya sobre una propuesta que ya existe. - *Hecho:* `criteria.yaml` con reglas en el paso 8 y `dev.on: false`.
6. **Registro completo.** El builder, el autopilot y la WFM apuntan su fila solos. - *Porqué:* el Sharpe desinflado solo vale si cuenta todas las pruebas. - *Coste:* M. - *Hecho:* una corrida nueva deja una fila por paso, incluido el 6, sin `backfill`.
7. **Población nula de punta a punta.** Plantilla de entrada aleatoria por toda la cadena, en dos activos. - *Porqué:* dice cuántas estrategias sin edge llegan al final. Hay que hacerlo antes de mirar supervivientes reales, no después. - *Coste:* L en sesiones y varias noches de CPU. - *Hecho:* una tabla «de N monos llegan K al paso 20», y los umbrales fijados contra ella.
8. **Reglas para 10, 12, 14, 16 y definición del paso 20**, con los umbrales movidos a `ledger/thresholds.yaml`. - *Coste:* M, más tus decisiones. - *Hecho:* ningún paso que juzga tiene `rules: []`; el limbo y el hecho ausente tienen destino explícito.
9. **Recalcular los nulos en la escala nueva de la t** (`OPEN #91`) y resolver la curva de mínimos (`OPEN #88`) en los estudios que la diferencian. - *Coste:* M. - *Hecho:* #91 y #88 cerrados.

Fase 2 — Que la fábrica corra sola en esta máquina (2-3 semanas)

10. **Endurecer el autopilot.** Límite de tiempo y detección de muerte en la ruta en vivo, reintentos de fallos pasajeros, «cero supervivientes» como final limpio, códigos de salida veraces, comprobación previa de recursos.
 - *Porqué:* 18 de 20 lanzamientos no terminaron.
 - *Coste:* M.
 - *Hecho:* diez corridas seguidas sin intervención, cada una en `FIN` o en un veredicto.
11. **Cola persistente y despachador** como servicio del sistema, con estado propio por corrida.
 - *Porqué:* es la pieza que falta para encadenar trabajos, y la base del segundo servidor.
 - *Coste:* M-L.

- *Hecho*: se encolan cinco proyectos por la noche y por la mañana hay cinco resultados, con el demonio reiniciado a mitad.

12. Avisos al móvil y resumen diario.

- *Coste*: M.
- *Hecho*: recibes un mensaje al acabar cada trabajo, uno por fallo duro, y uno cada mañana.

13. Autopilot fase 2 (16.5 → 20) y fase 3 (21 → 26), con la versión con stop como identidad propia y la verificación de MT5 por lotes.

- *Porqué*: hoy la cadena se queda a diez pasos del final.
- *Coste*: L.
- *Hecho*: un proyecto entra en el paso 6 y sale con un veredicto en el 26, sin tocar nada.

14. Registro de estrategias y pool validado.

- *Coste*: M. Requiere diagnosticar antes el descuadre de Hantec.
- *Hecho*: una tabla por identidad con su estado, y el pool de cada empresa escrito por el paso 26.

15. Un solo orquestador y guardas en código. Retirar `pipeline/run.py` y la cadena de la ventana, o hacer que llamen al autopilot; `core.worker.call` rechaza `count` sobre un custodio en corrida; las pruebas completas cada noche.

- *Coste*: M.
- *Hecho*: un botón «Autopilot» en la ventana y ninguna otra ruta de lanzamiento.

Fase 2B — En paralelo, porque no depende de lo anterior: ver las cuentas que ya tienes

16. Telemetría de la VPS, solo lectura. Servicio MQL5, colector y zona «Flota».

- *Porqué*: ya hay 4-5 terminales operando. Es valor desde el primer día y es la base de todo el seguimiento posterior.
- *Coste*: M. Necesita acceso a la VPS y Tailscale, que son tuyos.
- *Hecho*: la ventana enseña equity, flotante y distancia a los suelos de cada cuenta, y avisa si un terminal calla cinco minutos.

Fase 3 — Que busque sola (1-2 semanas)

17. Primera corrida real de «Investigar», de la propuesta al veredicto.

- *Coste*: M, más 10-30 \$ de tokens por propuesta.
- *Hecho*: una propuesta en disco, tres proyectos corridos, tres veredictos en la memoria.

18. Lazo cerrado con tope de gasto. Temporizador, política fija de `Test_`, ideas con dudas aparcadas, veredictos de vuelta al tablero, registro unido a la memoria.

- *Porqué*: es lo que convierte la fábrica en búsqueda sobre los 17 activos sin tocar.
- *Coste*: M.
- *Hecho*: una semana sin que pulses nada, con la cola siempre ocupada y el gasto bajo el tope.

19. Agentes fuera de tu sesión. Credencial de servicio, binario propio, temporizadores del sistema con aviso de fallo, tokens por corrida en el log, `fixer` y auditor solo cuando algo cambió.

- *Coste*: M. La credencial de API es gasto: decisión tuya.
- *Hecho*: los agentes corren con VS Code cerrado y sabes cuánto cuesta cada uno.

Fase 4 — Fondeo: de superviviente a cuenta (3-4 semanas)

Empieza cuando el pool tenga miembros. Si la fase 3 no los da, esta fase espera y la pregunta pasa a ser de investigación, no de ingeniería.

20. F3 y F4 de la cartera.

- *Coste*: L. Requiere la respuesta a Q11.
- *Hecho*: una cartera con su probabilidad de aprobar y su veredicto fuera de muestra.

21. F5 y el barrido de planes. Valor esperado por plan, complementos y oferta, conectado a `deals.worth`.

- *Porqué*: contesta qué reto comprar con tus 1.000 €.
- *Coste*: L. Requiere el resto del encargo 33 \$6.
- *Hecho*: un aviso «compra este plan, con esta cartera, valor esperado X».

22. Exportador de cartera a EAs. Magic, riesgo por tamaño de cuenta, filtro de noticias, comentario, y el paquete por cuenta.

- *Coste*: M. Incluye ver el filtro de noticias cruzar una noticia real en demo.
- *Hecho*: de una cartera sale una carpeta por cuenta lista para montar.

23. Pestaña de cartera en la ventana.

- *Coste*: M.

Fase 5 — Seguimiento en vivo contra el backtest (2 semanas)

24. Conciliación semanal por magic contra el retest de SQX.

- *Coste*: M. Requiere el 16 y el 22.
- *Hecho*: cada sábado, una tabla de operaciones casadas y sin casar por estrategia.

25. Deriva y criterios de retirada.

- *Coste*: M, más tus umbrales.
- *Hecho*: una estrategia fuera de su banda genera un aviso con la recomendación.

26. Despliegue asistido a la VPS.

- *Coste*: M.
- *Hecho*: apruebas y el paquete queda montado en el terminal correcto.

Fase 6 — El segundo servidor (2 semanas)

27. Modelo de nodos: los seis puntos de \$6.8.

- *Porqué aquí*: con la cola y los avisos hechos, es añadir un nodo. Antes sería duplicar una fábrica que no filtra.
- *Coste*: L.
- *Hecho*: la cola reparte entre dos servidores y la «Torre» enseña las instalaciones de ambos.

Si el segundo servidor ya está contratado y parado, adelanta solo dos cosas: úsalo como destino de la copia (punto 3) y para la población nula (punto 7), que es CPU pura.

Fase 7 — Darwinex

28. El camino de cuenta real (M3-M9 de `portfolio/PLAN.md`): pesos, veredicto y OOS en papel.

- *Coste*: L. Reutiliza la telemetría y la conciliación de las fases 2B y 5.

9 · Lo que solo puedes decidir tú

Decisiones de una vez. Las cinco primeras bloquean la fase 1.

1. **Las cifras de `_study.yaml`** y, dentro de ellas, el conflicto con tu regla: el suelo propuesto es de 20-30 operaciones/año y tú pediste 40-50. La calibración dice que subir de 20 a 50 baja el rendimiento de 31,7 a 8,6 por mil. Una de las dos cede.
2. **La regla del paso 8** y sobre qué t juzga (el equipo rojo propone `drift_excess_t`, $\text{limbo} \geq 1,65$ y pasa $\geq 2,33$).
3. **Qué es «pasar» el paso 20**: unanimidad, sin fallo, o solo el StepM.
4. **Si la reserva de `oos2` ata al autopilot**. Recomendando que sí: es justo el caso para el que se escribió.
5. **Si se corre la población nula antes de seguir**. Recomendando que sí.
6. **Cartera**: Q10, Q11, Q16 y `DECISIONS.md` #2, #5, #12.
7. **Fondeo**: el resto del encargo 33 §6 (riesgo por fase, recompra, qué `h`), y las preguntas a soporte de FTMO, Hantec, FundedNext y FundingPips.
8. **Infraestructura**: el cortafuegos, el canal de avisos, la credencial de servicio para los agentes, el acceso a la VPS y el destino de la copia.

10 · Lo que no se verificó

- **Si la API de SQX es alcanzable desde internet.** Vi los puertos en `0.0.0.0` y la IP pública; no pude leer el cortafuegos ni sé si tu proveedor filtra por delante.
- **Que las zonas de la ventana funcionen.** No la arranqué; «funciona» significa construido y con pruebas.
- **El estado de las pruebas hoy.** No corrí `checks.py` ni `pytest`, porque escriben cachés. Las cuatro que fallaban son del informe del 29 de septiembre.
- **Las cifras de calibración** (87.357 → 49) vienen de `AlgoData/scratch/calib_final/` y del informe del equipo rojo; no las recalculé.
- **Qué EAs corren hoy en la VPS** y de dónde salieron. Deduzco que son los hechos a mano de antes del workflow; ningún documento lo dice.
- **El segundo servidor.** No sé si existe, qué sistema lleva ni cuánta máquina es.
- **Lo que gastan los agentes en tokens.** Ningún log lo apunta.
- **Otra sesión estaba trabajando mientras escribía esto** (un A/B de filtros de build en el custodio, a las 04:22). Sus cambios de esta madrugada pueden no estar reflejados.
- Los costes del plan son estimaciones mías, sin medir.